

Practice: Two-Sum and K-Sum Variants

Warm-up — Two Sum

Prompt:

Given an integer array `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`. You may assume exactly one solution and you may not use the same element twice. Return indices in any order.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- Values fit in 32-bit integers

Hint:

Use a hashmap storing value→index. For each value, check if `target - value` is in the map (single pass, $O(n)$ time, $O(n)$ space).

Example:

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

Two Sum II — Input Array Is Sorted

Prompt:

Given a 1-indexed sorted array of integers and a `target`, return the indices (1-based) of the two numbers such that they add up to `target`. Exactly one solution.

Hint:

Use two pointers `l` and `r`. Move `l` forward if `sum < target`, `r` backward if `sum > target`. $O(n)$ time, $O(1)$ space.

3Sum

Prompt:

Given an array `nums`, find all unique triplets `[a,b,c]` such that `a + b + c == 0`. Return no duplicate triplets.

Hint:

Sort the array, fix one index `i`, reduce to two-sum for the remainder using two pointers. Skip duplicates at `i` and inside the two-pointer loop.

Subarray Sum Equals K

Prompt:

Given an integer array `nums` and an integer `k`, return the count of contiguous subarrays whose sum equals `k`.

Hint:

Use prefix sums and a hashmap that counts how many times each prefix sum has occurred. For current prefix `S`, add `count[S - k]` to answer.

4Sum

Prompt:

Given an array `nums` and `target`, find all unique quadruplets `[a,b,c,d]` such that they sum to `target`. No duplicate quadruplets.

Hint:

Sort and either use two nested loops + two pointers, or implement recursive `k`-sum that reduces to two-sum. Carefully skip duplicates and prune sums that can't reach target.

Two-Sum Variants (Closest / Count Pairs < K)

Prompt A (Closest):

Given `nums` and `target`, find two numbers whose sum is closest to `target` and return their sum (or pair).

Hint A:

Sort + two pointers; track minimum absolute difference.

Prompt B (Count < K):

Count number of pairs whose sum is strictly less than `K`.

Hint B:

Sort + two pointers; for each left, move right to count how many rights form valid pairs in $O(n)$ time.

Two Sum on Other Data Structures (BST / Linked List)

Prompt:

Given a BST (or singly linked list), determine if there exist two nodes whose values sum to `k`. Return boolean.

Hint:

Option 1: DFS + hash set recording seen values, check complement.

Option 2 (BST): use two iterators (inorder and reverse inorder) to simulate two-pointer on sorted sequence with $O(h)$ extra space.

K-Sum Challenge (Generalized)

Prompt:

Implement a general `kSum(nums, target, k)` that returns unique k-tuples summing to target.

Hint:

Sort nums. Recurse: for $k > 2$ loop index i and call $(k-1)$ Sum on remainder; base case $k == 2$ use two-pointer two-sum. Prune when $\text{smallest} * k > \text{target}$ or $\text{largest} * k < \text{target}$ to speed up.

Practice Tips

- Start with the hashmap approach for Two Sum to internalize complements.
- Practice sorted two-pointer reductions next (Two Sum II, 3Sum, 4Sum).
- Implement generalized k-sum to solidify recursion + pruning patterns.
- Time yourself: aim for $O(n)$ or $O(n \log n)$ solutions where possible.

End of practice sheet